

Late Breaking Results: Language-level QoR modeling for High-Level Synthesis

Dimosthenis Masouros^{*}, Aggelos Ferikoglou^{*}, Georgios Zervakis[†], Sotirios Xydis^{*} and Dimitrios Soudris^{*}

^{*}Microprocessors and Digital Systems Laboratory, ECE, National Technical University of Athens, Greece

[†]Department of Computer Engineering and Informatics, University of Patras, Greece

ABSTRACT

This paper proposes a language-level modeling approach for High-Level Synthesis based on the state-of-the-art Transformer architecture. Our approach estimates the performance and required resources of HLS applications directly from the source code when different synthesis directives, in terms of HLS *#pragmas*, are applied. Results show that the proposed architecture achieves 96.02% accuracy for predicting the feasibility class of applications and an average of 0.95 and 0.91 R^2 scores for predicting the actual performance and required resources, respectively.

ACM Reference Format:

Dimosthenis Masouros^{*}, Aggelos Ferikoglou^{*}, Georgios Zervakis[†], Sotirios Xydis^{*} and Dimitrios Soudris^{*}. 2024. Late Breaking Results: Language-level QoR modeling for High-Level Synthesis. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3649329.3663500>

1 INTRODUCTION

Language Models have been widely adopted lately for modeling computer-related language tasks (e.g., GitHub’s Copilot offers auto-complete suggestions as you code), opening up new possibilities for their application in domain-specific contexts. In FPGA programming, particularly with High-Level Synthesis (HLS), this shift allows to leverage LLMs to simplify hardware design complexities. By offering real-time insights during coding, LLMs can enable “on-the-fly” assessment and optimization of HLS applications, further raising the design abstraction. NVIDIA’s introduction of ChipNeMo, a domain-adapted LLM, further supports this trend, serving as designers’ personal assistant for chip design tasks [5].

In HLS, developers optimize FPGA execution using *#pragmas* (e.g., pipeline) and constraints to guide hardware synthesis. However, accurately estimating the impact of each *#pragma* on Quality-of-Results (QoR) for the final FPGA design is challenging. Ideally, characterizing each design point through synthesis, place-and-route, and direct FPGA execution is preferred but time-consuming due to compilation and mapping complexities. Prior research on

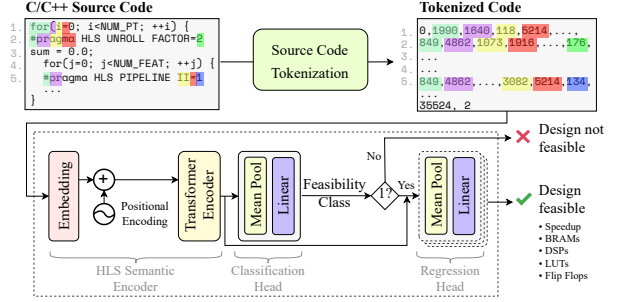


Figure 1: Proposed Transformer-based Neural Architecture.

HLS performance modeling faces limitations: some rely on simplifications and assumptions about technology and hardware [4], while others depend on the intermediate representation (IR) of the source code and may not capture effects of pre-processor *#pragmas* [3, 6, 7, 9], often requiring time-consuming software stacks for predictions. Towards a more “agile hardware development process”, the goal is to enable QoR prediction and design feasibility at higher levels of the stack, ideally directly from the source code, thus, allowing developers to get live optimization recommendations as they code.

In this paper, we propose a language-level performance and resource modeling approach for HLS based on the state-of-the-art Transformer architecture. To the best of our knowledge, this is the first work to approach QoR modeling for HLS directly from the source-code, without i) the expensive fine-tuning of pre-trained LLMs [2] and ii) the need for any structural information from the compiler stack like current GNN-based State-of-the-Art (SotA) [3, 6]. Results show that our architecture achieves $\approx 96\%$ accuracy on predicting whether a *#pragma*-optimized HLS application will fit into the available resources of the target device and, if it does, an average of $\approx 0.91 R^2$ score for predicting its actual QoR metrics.

2 LANGUAGE-LEVEL HLS MODELING

Figure 1 shows an overview of the proposed approach. Given a C/C++ source code with a set of HLS *#pragmas* applied to it, our goal is i) to determine if the design is feasible, ensuring that none of the resources (QoR metrics) exceed the available resources of the FPGA device (classification task); and ii) to predict the speedup of the application compared to the non-*#pragmas* version, as well as its resource utilization on the device (regression tasks).

Source Code Tokenization: The tokenizer converts the input C, C++ source code along with its applied HLS *#pragmas* into a sequence of tokens or subword units that can be used as input for the Transformer model. We utilize Codebert’s [8] pre-trained tokenizer, which uses byte-level Byte-Pair-Encoding, a subword tokenizer

This work has been partially supported by the EU Horizon Program CONVOLVE under Grant Agreement No: 101070374.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06

<https://doi.org/10.1145/3649329.3663500>

Task	R^2	MAE
Speedup	0.958	0.84
BRAM	0.960	3.38
DSP	0.923	4.13
FF	0.894	5.05
LUT	0.842	11.7
Feasibility	96.02%	

Table 1: Accuracy / score per prediction task.

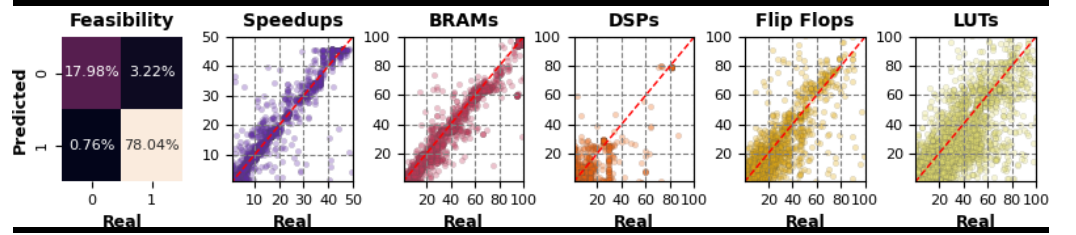


Figure 2: Confusion Matrix (classification task) and Real versus Predicted values for the five target QoR metrics (regression task)

technique, that breaks text into smaller subword units. This allows models to handle a wide variety of languages and create a more flexible vocabulary. The top part of Figure 1 shows a simple tokenization example for a nested loop source code.

Proposed Architecture: The proposed architecture utilizes the Transformer Encoder, excluding the decoder for regression objectives, which focus solely on sequence generation. Task-specific heads transform encoded source code features into the target prediction variables. The *HLS semantics encoder* comprises an embedding layer, positional encoder, and transformer encoder, extracting informative features from complex HLS code. The *classification head* determines application feasibility for the target FPGA, employing mean pooling, linear layers, ReLU, and Dropout for non-linear behavior capture. Last, the *Regression heads* predict QoR metrics only when feasibility is confirmed, sharing the same architecture as the classification head. There are five distinct regression heads for predicting Speedup, BRAMs, DSPs, LUTs, and Flip Flops.

Model Training: We adopt a learning rate warm-up and decay approach to stabilize early training and enhance Transformer performance. Beginning with an initial rate of $2e-4$, we linearly increase this value over the first 10 epochs to adapt the model gradually. Subsequently, we employ a learning rate decay strategy by multiplying the rate by a factor of $\gamma = 0.9$. For loss calculation, we employ a multitask approach combining Binary Cross Entropy for classification tasks and Mean Squared Error for regression. During training, the loss function evolves, initially distributing equal weight to all tasks before gradually shifting focus towards the main prediction task. We introduce a weight factor (w) that increases as we near the total number of epochs, with 25 epochs identified as a pivotal point for adjusting the loss calculation.

3 EVALUATION

We train our language-based model using a dataset linking over 100 C/C++ source codes with their corresponding QoR metrics for various HLS directives, similar to the approach followed in [1].

Model Accuracy: We evaluate our model's performance in predicting feasibility class and QoR metrics using a 4-fold cross-validation approach on 80% of our dataset, reserving 20% for testing. The model is assessed based on Mean Absolute Error (MAE) and coefficient of determination (R^2). Table 1 shows that in classification, our model achieves 96% accuracy, with a negligible false negative rate. For regression, R^2 values range from 0.84 to 0.96, with MAE ranging from 0.84 to 11.7. While speedup and BRAMs show strong alignment

	Speedup	BRAM	DSP	FF	LUT	Feasibility
R^2	0.085	0.46	0.039	0.072	0.066	83.19%
MAE	6.95	18.32	10.23	13.27	22.77	

Table 2: Predictions with Codebert [8] as semantic encoder

with the 45° regression line, DSPs, Flip Flops, and LUTs exhibit more variability, yet without systematic over- or under-estimation.

Impact of HLS Semantic Encoder: We evaluate the effectiveness of our HLS domain-specific encoder (§2) against general-purpose pre-trained LLMs. Adopting CodeBert [8] as a substitute for the HLS semantic encoder while retaining the classification and regression heads, we explore whether a pre-trained LLM can adequately capture HLS-relevant features. Our results, as summarized in Table 2, indicate that while the CodeBert variant achieves approximately 89% accuracy in predicting feasibility class, its performance in predicting QoR metrics is notably poorer. With R^2 values ranging from 0.04 to 0.46 and MAE values from 6.9 to 22.7, it shows the necessity of domain-specific language models for these tasks.

4 CONCLUSION

We introduced a domain-specific, language-level modeling approach tailored for HLS. Our approach predicts the feasibility class and QoR metrics of applications with different HLS *#pragmas*. Results demonstrate an accuracy of $\approx 96\%$ for predicting the feasibility class of applications and an average of ≈ 0.91 R^2 score for QoR prediction, thereby significantly enhancing the HLS design process.

REFERENCES

- [1] Ferikoglou Aggelos et al. 2023. CollectiveHLS: Ultra-fast Knowledge-Based HLS Design Optimization. *IEEE Embedded Systems Letters* (2023).
- [2] Yunsheng Bai et al. 2023. Towards a Comprehensive Benchmark for High-Level Synthesis Targeted to FPGAs. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [3] Ustun Ecenur et al. 2020. Accurate operation delay prediction for FPGA HLS using graph neural networks. In *39th Intl. conference on computer-aided design*.
- [4] Zhong Guanwen et al. 2016. Lin-analyzer: A high-level performance analysis tool for FPGA-based accelerators. In *Proceedings of 53rd Design Automation Conference*.
- [5] Mingjie Liu et al. 2023. Chipnemo: Domain-adapted llms for chip design. *arXiv preprint arXiv:2311.00176* (2023).
- [6] Ferretti Lorenzo et al. 2022. Graph neural networks for high-level synthesis design space exploration. *ACM Trans. on Design Automation of Electronic Systems* (2022).
- [7] Wu Nan et al. 2022. IronMan-Pro: Multiobjective Design Space Exploration in HLS via Reinforcement Learning and Graph Neural Network-Based Modeling. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 3 (2022), 900–913.
- [8] Feng Zhangyin et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [9] Lin Zhe et al. 2020. HL-Pow: A learning-based power modeling framework for high-level synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE.